

A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a neural network diagram.

Debugging

COMP211 Connor McMahon

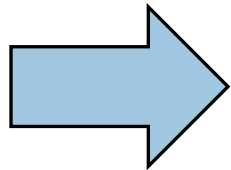
Stepping through an Input-Handling Program in C Using VSCode

- We'll be walking through how to debug the following program in VSCode.
- (Note there is no bug in this program)

```
#include <stdio.h>
int main() {
    int my_char;
    while ((my_char = getchar()) != EOF) {
        if (my_char >= 'a' && my_char <= 'z') {
            my_char = my_char - ('a' - 'A');
        }
        putchar(my_char);
        fflush(stdout);
    }
}
```

Debugging an Input-Handling Program in C Using VSCode

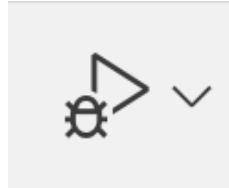
- Place a breakpoint in your program by clicking to the left of the line number (this area is known as the **gutter**) where you would like to place the breakpoint. A red dot should appear.



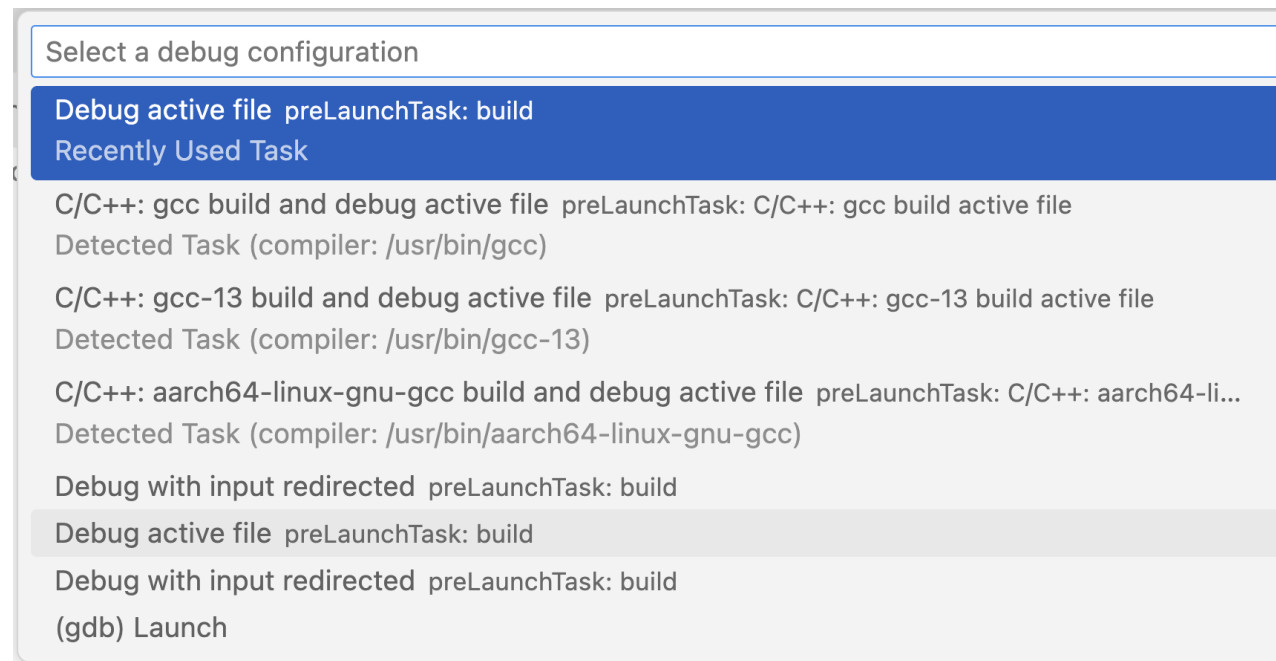
```
1  #include <stdio.h>
2  int main() {
3      int my_char;
4      while ((my_char = getchar()) != EOF) {
5          if (my_char >= 'a' && my_char <= 'z') {
6              my_char = my_char - ('a' - 'A');
7          }
8          putchar(my_char);
9          fflush(stdout);
10     }
11 }
```

Launching the Debugger

- In the upper right-hand corner, click the debug button.



- Then select **Debug active file**

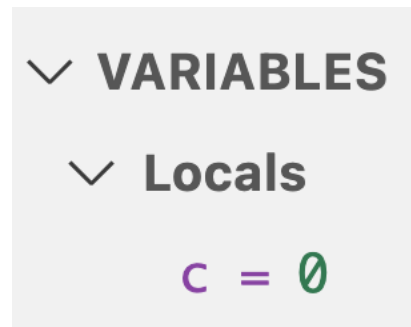


Debugging an Input-Handling Program in C Using VSCode

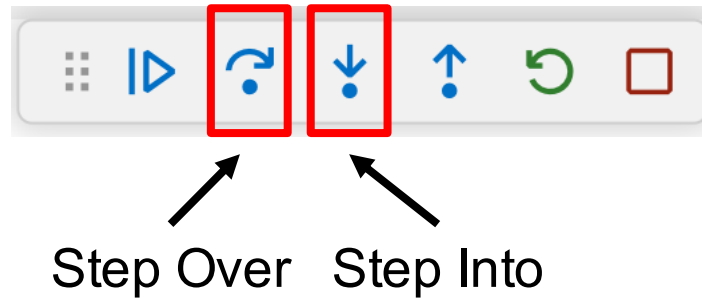
- This panel should appear. You'll use this to control stepping through the program.



- Notice on the left panel that you can see the value of the local variable.

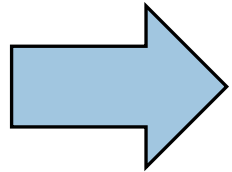


Step Over vs Step Into



	Step Over	Step Into
Definition	Executes the current line of code completely but skips stepping into any functions that are called on that line.	Executes the current line and pauses inside any function that is called, allowing you to debug the function line by line
Use Case	When you are not interested in the internal details of a function and just want to proceed to the next line of the current function.	When you want to examine the internal workings of a function being called.
What Happens	If the line contains a function call, the function will execute entirely instead of stepping through the internal lines of the function.	The debugger pauses at the first line of the function being called. You can then continue to step through the function's execution line by line.

Debugging an Input-Handling Program in C Using VSCode



```
1  #include <stdio.h>
2  int main() {
3      int my_char;
4      while ((my_char = getchar()) != EOF) {
5          if (my_char >= 'a' && my_char <= 'z') {
6              my_char = my_char - ('a' - 'A');
7          }
8          putchar(my_char);
9          fflush(stdout);
10     }
11 }
```

Debugging an Input-Handling Program in C Using VSCode

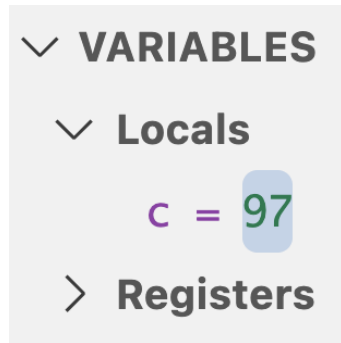
- We don't have any need to step into `getchar()`, so we will click step-over.
- Notice that the program does not proceed. This is because `getchar()` is waiting to read input. In the terminal, type the letter `a` and click enter.



- Your program should have now moved onto line 5 of `main`.

Debugging an Input-Handling Program in C Using VSCode

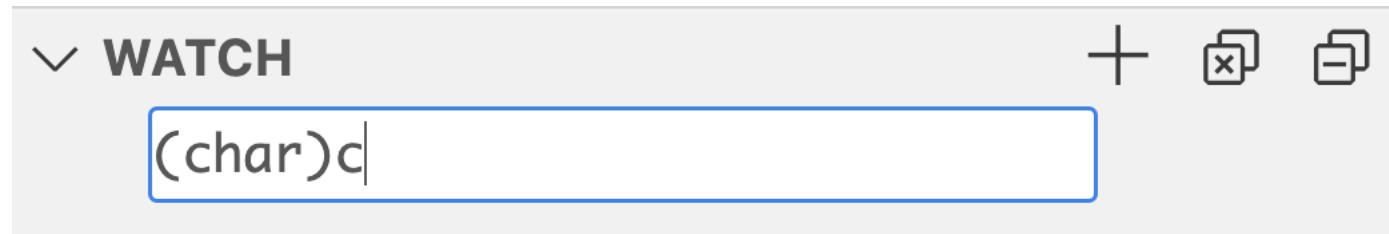
- If you examine the variables panel, you'll notice that the value of **c** has been updated to **97**. Since **c** is of type int, it is displayed in decimal format.



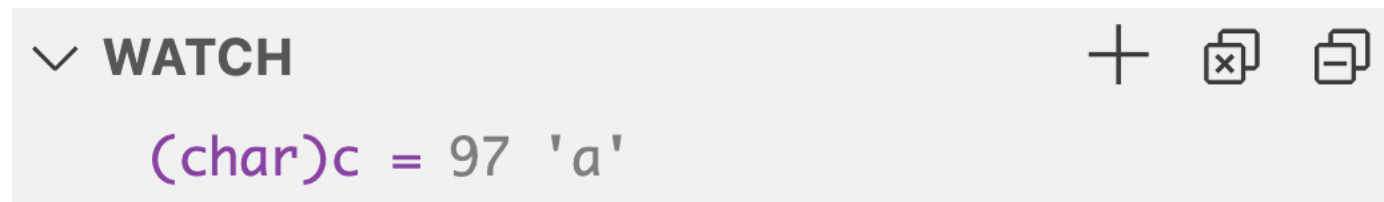
- Unless you have all of the ASCII values memorized, this value is not very useful to us. On the next slide, we'll see how to view the **char** value of c

Debugging an Input-Handling Program in C Using VSCode

- To view the ASCII value of `c`, go to the **Watch panel** (located just below the **Variables panel**) and click the plus icon. In the input box, cast `c` to the `char` type and press Enter.
 - A watchpoint allows you to monitor a specific variable or memory location during program execution.



- You should now see the char value of `c`



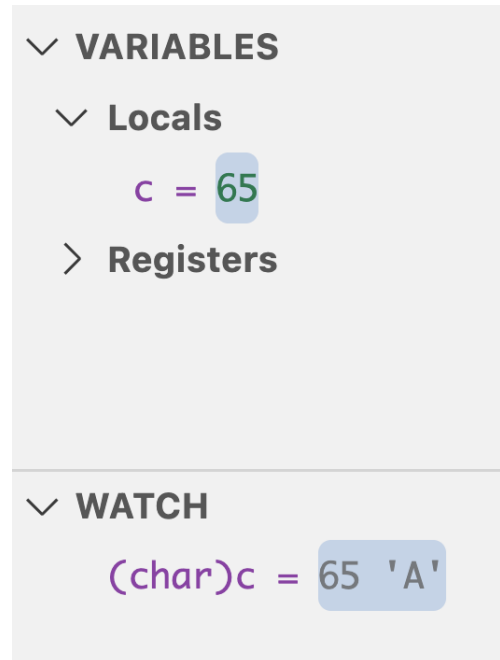
Debugging an Input-Handling Program in C Using VSCode

- Since we entered a lowercase letter, we can expect this if condition to evaluate to true.
- If we step again, the debugger will move into the if statement.

```
1  #include <stdio.h>
2  int main() {
3      int my_char;
4      while ((my_char = getchar()) != EOF) {
5          if (my_char >= 'a' && my_char <= 'z') {
6              my_char = my_char - ('a' - 'A');
7          }
8          putchar(my_char);
9          fflush(stdout);
10     }
11 }
```

Debugging an Input-Handling Program in C Using VSCode

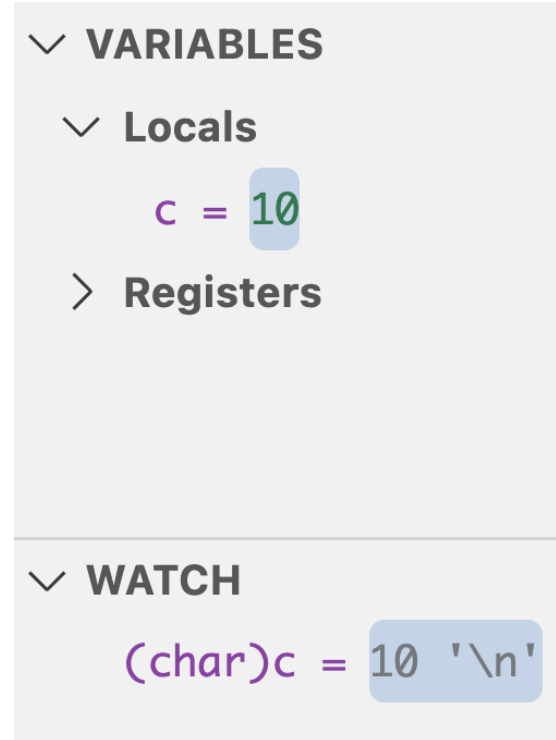
- Step again (step over and step into will be the same in this case). The variable **c** now stores an upper-case **A**.



- Step over `putchar()` and `fflush(stdout)`. A capital **A** should have been printed to your terminal.

Debugging an Input-Handling Program in C Using VSCode

- On this next iteration, the loop will be processing the **newline** character that you entered earlier.



- Step all the way through this iteration of the loop and see the newline character get printed to your terminal.

Debugging an Input-Handling Program in C Using VSCode

- When you get back to the top, enter a longer string (e.g. “HeLlo”).
- Step through the program with this longer input to get more practice.
- What happens if you don't include **fflush()** when debugging?
 - Try it out!

Input Buffering

- Input from stdin is stored in a buffer in memory before it is passed to your program.
- When you type data at the terminal, it is not immediately delivered to the program. Instead, it waits in the buffer until:
 - The user presses **Enter**.
 - The buffer is full

Output Buffering

- The text you send (e.g., using `printf`) is temporarily stored in a **buffer**.
- The buffer is flushed (output is sent to the terminal) when
 - A newline character (`\n`) is encountered.
 - The buffer is full.
 - The program explicitly flushes it using `fflush(stdout)`.

Output buffer

Apples\n

Debugging an Input-Handling Program in C Using VSCode

- What happens if you don't include `fflush()` when debugging?
 - Remember `printf` uses buffered I/O, meaning that the text you print is stored in an internal buffer and is only sent to the terminal when the buffer is flushed (this happens when a newline is printed, the buffer fills up, or the program ends).
 - If you don't call `fflush(stdout)` while debugging, you may not see your program's output immediately as you step through the code. This makes it harder to observe intermediate states.
 - Calling `fflush(stdout)` forces the buffer to be written out right away, making it easier to follow the program's execution.

How is the debugger launched?

- When you launch the debugger in VS Code, VS Code reads the `.vscode/launch.json` file.
- There are two debugging options:
 - "Debug active file"
 - "Debug with input redirected"

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug active file",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}",
      "cwd": "${fileDirname}",
      "stopAtEntry": false,
      "externalConsole": false,
      "MIMode": "gdb",
      "preLaunchTask": "build"
    },
    {
      "name": "Debug with input redirected",
      "type": "cppdbg",
      "request": "launch",
      "program": "/bin/bash",
      "args": [
        "-lc",
        "\"${fileDirname}/${fileBasenameNoExtension}\" < <
        \"${fileDirname}/test\""
      ],
      "cwd": "${fileDirname}",
      "stopAtEntry": false,
      "externalConsole": false,
      "MIMode": "gdb",
      "preLaunchTask": "build"
    }
  ]
}
```

Hidden Files

- A **hidden file** is simply a file that is normally not shown when you list the contents of a directory.
- A file or directory is hidden when its name begins with a period (.). For example:
 - `.vscode`
 - `.git`
 - `.gitignore`
 - `.bashrc`
- Hidden files are commonly used for configuration and program metadata that you usually don't need to interact with during normal work.

tasks.json

- Before the debugger configuration starts, VS Code runs the task specified by:
 - "preLaunchTask": "build"
- The "build" task is defined in .vscode/tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build",
      "type": "shell",
      "command": "gcc",
      "args": [
        "-fdiagnostics-color=always",
        "-g",
        "-Wall",
        "-Wextra",
        "-Wpedantic",
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": "$gcc",
      "group": {
        "kind": "build",
        "isDefault": true
      }
    }
  ]
}
```

tasks.json

```
"command": "gcc",  
"args": [  
    "-fdiagnostics-color=always",  
    "-g",  
    "-Wall",  
    "-Wextra",  
    "-Wpedantic",  
    "${file}",  
    "-o",  
    "${fileDirname}/${fileBasenameNoExtension}"  
],
```

`${file}` represents the currently active file in VS Code.

`${fileDirname}` represents the directory containing the active file.

`${fileBasenameNoExtension}` represents the active file's filename without its extension.

tasks.json

- For example, if the active file is:

```
/home/student/project/main.c
```

- then the variables expand to:

```
${file} = /home/student/project/main.c
```

```
${fileDirname} = /home/student/project
```

```
${fileBasenameNoExtension} = main
```

- Therefore, the build command is effectively:

```
gcc -fdiagnostics-color=always -g -Wall -Wextra -Wpedantic  
/home/student/project/main.c -o /home/student/project/main
```

- This creates an executable with the same base name as the source file. For example:
 - With source file `main.c`, an executable named `main` is created

Launch.json

- After the build finishes, the "Debug active file" configuration runs that executable using:

```
"program": "${fileDirname}/${fileBasenameNoExtension}"
```

- So, for `main.c`, VS Code runs:

```
/home/student/project/main
```

Launch.json

- `"type": "cppdbg"` tells VS Code which debugger extension should handle this launch configuration.
- `cppdbg` is the debug type provided by Microsoft's **C/C++ extension**. It tells VS Code to use the C/C++ debugging system for this configuration.
- The C/C++ debugging extension acts as a **debug adapter** to communicate with the actual debugger (GDB).

Launch.json

- `"MIMode" : "gdb"` tells the C/C++ debug adapter that the underlying debugger it should communicate with is GDB.
- The "MI" part stands for Machine Interface. GDB has a machine-oriented interface that programs such as VS Code can use to control it.
- Importantly, **GDB is the thing actually debugging your program.** VS Code provides the interface you interact with, while cppdbg acts as the middle layer that allows VS Code to control GDB.

Debugging an Input-Handling Program in C Using VSCode

- We'll be walking through how to debug the following program in VSCode.

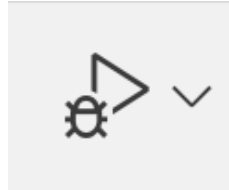
```
#include <stdio.h>
int main() {
    int my_char;
    while ((my_char = getchar()) != EOF) {
        if (my_char >= 'a' && my_char <= 'z') {
            my_char = my_char - ('a' - 'A');
        }
        putchar(my_char);
        fflush(stdout);
    }
}
```

Create your input file

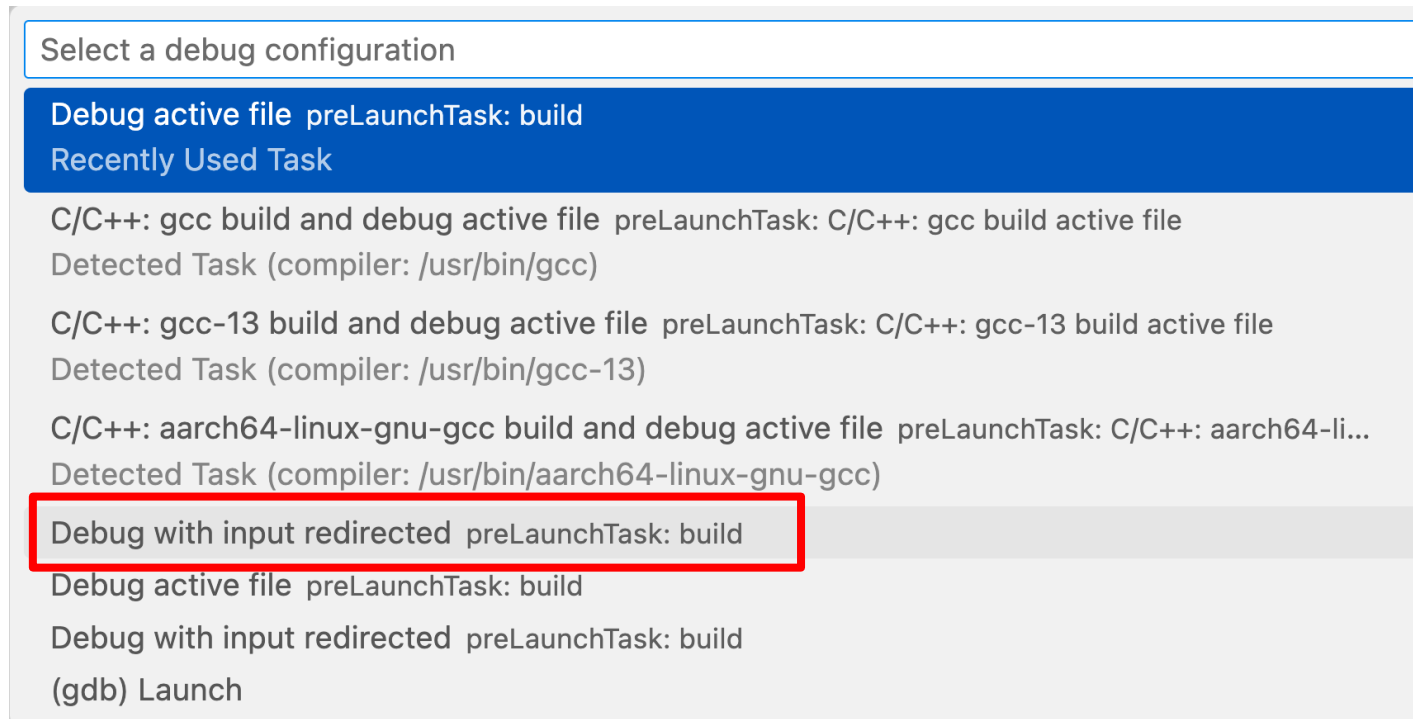
- In the same directory as `input.c`, create a file named `test`.
- Write a single line to the file
 - E.g. Hello!

Launching the Debugger with Input Redirected from a File

- In the upper right-hand corner, click the debug button.



- Then select **Debug with input redirected**



Stepping through with stdin redirected

- As you step through, notice that the program is reading from your file “test” instead of reading input from the terminal.
- How did it know to read from test?
- From launch.json!

How is the debugger launched?

- When you launch the debugger in VS Code, VS Code reads the `.vscode/launch.json` file.
- There are two debugging options:
 - "Debug active file"
 - "Debug with input redirected"

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Debug active file",
      "type": "cppdbg",
      "request": "launch",
      "program": "${fileDirname}/${fileBasenameNoExtension}",
      "cwd": "${fileDirname}",
      "stopAtEntry": false,
      "externalConsole": false,
      "MIMode": "gdb",
      "preLaunchTask": "build"
    },
    {
      "name": "Debug with input redirected",
      "type": "cppdbg",
      "request": "launch",
      "program": "/bin/bash",
      "args": [
        "-lc",
        "\"${fileDirname}/${fileBasenameNoExtension}\" < <
        \"${fileDirname}/test\""
      ],
      "cwd": "${fileDirname}",
      "stopAtEntry": false,
      "externalConsole": false,
      "MIMode": "gdb",
      "preLaunchTask": "build"
    }
  ]
}
```

tasks.json

- Before the debugger configuration starts, VS Code runs the task specified by:
 - "preLaunchTask": "build"
- The "build" task is defined in `.vscode/tasks.json`

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "build",
      "type": "shell",
      "command": "gcc",
      "args": [
        "-fdiagnostics-color=always",
        "-g",
        "-Wall",
        "-Wextra",
        "-Wpedantic",
        "${file}",
        "-o",
        "${fileDirname}/${fileBasenameNoExtension}"
      ],
      "options": {
        "cwd": "${fileDirname}"
      },
      "problemMatcher": "$gcc",
      "group": {
        "kind": "build",
        "isDefault": true
      }
    }
  ]
}
```

Debug with input redirected

- The "Debug with input redirected" configuration works slightly differently.
- Instead of launching the executable directly, it launches Bash:
- "program": "/bin/bash"
- And gives bash the command circled

```
"name": "Debug with input redirected",  
"type": "cppdbg",  
"request": "launch",  
"program": "/bin/bash",  
"args": [  
    "-lc",  
    "\"${fileDirname}/${fileBasenameNoExtension}  
\" < \"${fileDirname}/test\""  
],  
"cwd": "${fileDirname}",  
"stopAtEntry": false,  
"externalConsole": false,  
"MIMode": "gdb",  
"preLaunchTask": "build"
```

Debug with input redirected

```
"args" : [
```

```
    "-lc",
```

```
    "\"${fileDirname}/${fileBasenameNoExtension}\" <
```

```
    \"${fileDirname}/test\" "
```

```
]
```

- After variable substitution, this might become:

```
"/home/student/project/main" < "/home/student/project/test"
```

- This runs the program while redirecting standard input from the file named test.

What is `-c` for?

- `-c` means execute a command
- Normally, if you run `/bin/bash`
 - Bash starts a shell and waits for you to type commands.

`/bin/bash -c "some command"` means

- Start Bash, execute "some command", and then exit.
- So your our configuration effectively does:
`/bin/bash -lc "\".../program\" < \".../test\""`

What about -l?

- `-l` tells Bash to behave as a **login shell**, which affects which shell startup files/environment it loads.

Debug with input from a different file

- Create a new file called `example.in` with a single line

```
learncli$ echo "Hello from example.in!" > example.in
```

- In `launch.json`, update the the name of the file

```
"args": [  
  "-lc",  
  "\"${fileDirname}/${fileBasenameNoExtension}\" <  
  \"${fileDirname}/example.in\""  
]
```

What if the file is in a different directory?

```
learncli$ mkdir -p one_dir/two_dir
```

```
learncli$ echo "new_example" > one_dir/two_dir/new_example
```

- In `launch.json`, update the the path and the name of the file

```
"args": [  
  "-lc",
```

```
    "\"${fileDirname}/${fileBasenameNoExtension}\" <
```

```
    \"${fileDirname}/one_dir/two_dir/new_example\" \"
```

```
]
```

```

#include <stdio.h>

int modifyBits(int x) {
    // Step 1: Set bit 3
    x = x | (1 << 3);

    // Step 2: Toggle bit 2
    x = x ^ (1 << 2);

    // Step 3: Clear bit 1
    // BUG: XOR toggles instead of clears
    x = x ^ (1 << 1);

    // Step 4: Set bit 0
    x = x | 1;

    return x;
}

```

```

int main(void) {
    int result;

    // Test 1
    result = modifyBits(2);
    printf("Test 1: expected 13, got %d\n", result);

    // Test 2
    result = modifyBits(6);
    printf("Test 2: expected 9, got %d\n", result);

    // Test 3
    result = modifyBits(0);
    printf("Test 3: expected 13, got %d\n", result);

    return 0;
}

```

Debugging Process

1. Run the tests to see which one is failing.

- Test 3 is failing

2. Then we will write down what we had expected would happen

Operation	Prediction	Actual
Initial value of num	0x0 = 0b0000_0000	
Set bit 3	0x8 = 0b0000_1000	
Toggle bit 2	0xC = 0b0000_1100	
Clear bit 1	0xC = 0b0000_1100	
Set bit 0	0xD = 0b0000_1101	

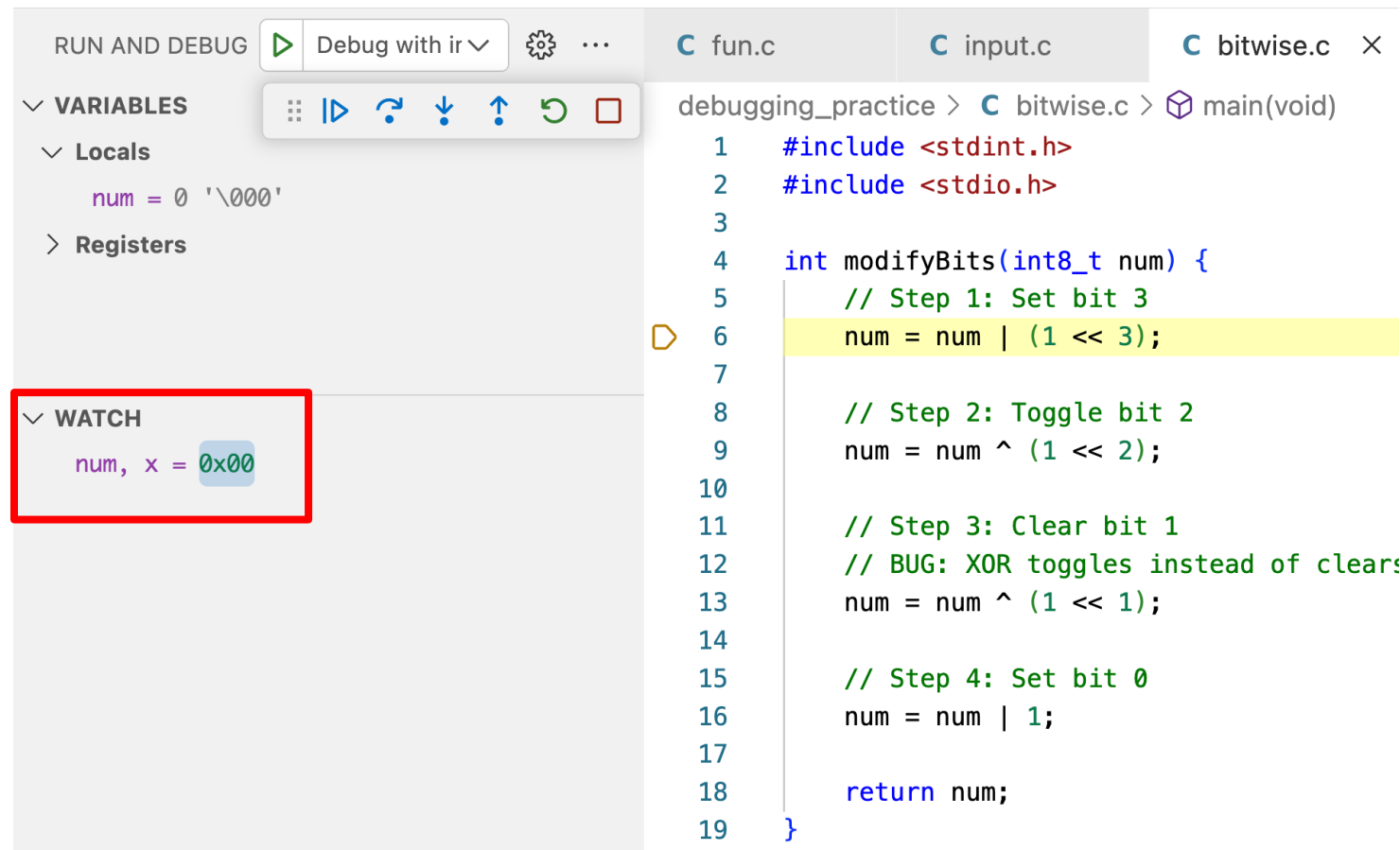
3. Set the breakpoint at the beginning of the failing test

4. Start debugging

- Trace through the code and find where the output doesn't match your prediction.

Debugging Process

- For this example, it is helpful if we can see the result in hex.
- Go to the watch panel, click the plus button and enter
 - num, x



Representing a Pixel

- A pixel's color can be represented using four channels: **red, green, blue, and alpha (RGBA)**. The red, green, and blue channels determine the color of the pixel, while the alpha channel determines its transparency.
- Each channel is an **8-bit value**, so it can range from 0 to 255 (0x00 to 0xFF). For example:
 - Red: 0xA5
 - Green: 0x3C
 - Blue: 0x72
 - Alpha: 0xFF
- Since each channel requires 8 bits, all four channels can be packed into a single **32-bit integer**:
 - 0xA53C72FF
- In this exercise, the channels are stored in **RGBA order**, with red in the most significant 8 bits and alpha in the least significant 8 bits.

```

#include <stdint.h>
#include <stdio.h>

uint32_t makeColor(uint8_t red, uint8_t green, uint8_t blue, uint8_t alpha) {
    uint32_t color = 0;

    // Put red in hexits 7-6
    color = color | ((uint32_t)red << 6);

    // Put green in hexits 5-4
    color = color | ((uint32_t)green << 4);

    // Put blue in hexits 3-2
    color = color | ((uint32_t)blue << 2);

    // Put alpha in hexits 1-0
    color = color | alpha;

    return color;
}

int main(void) {
    uint32_t result = makeColor(0xA5, 0x3C, 0x72, 0xFF);

    printf("Expected: 0xA53C72FF\n");
    printf("Actual: 0x%08X\n", result);

    return 0;
}

```

Buggy Program

Operation	Prediction	Actual
Put red in hexits 7-6		
Put green in hexits 5-4		
Put blue in hexits 3-2		
Put alpha in hexits 1-0		

Issues that can pop up

- If your file doesn't compile, the debugger won't start
- If you modify the file and don't restart the debugger, then you will be working off of the old file